

Funzioni SQL avanzate e operatori

UNION e UNION ALL:

- **UNION:** unisce colonne omogenee da tabelle diverse; ignora i criteri di join
- *Richiede stesso numero di colonne e tipi compatibili*
- **UNION ALL:** come UNION ma mantiene i duplicati
- Uso tipico: *aggregare dati da dataset separati in un'unica lista*

HAVING vs WHERE:

- **WHERE:** filtra sulle righe originali
- **HAVING:** filtra sui gruppi creati da GROUP BY
- **Esempio:** COUNT(città) per regione, poi HAVING > 10; posizione tra GROUP BY e ORDER BY

EXISTS:

- Ritorna true se la sottoquery restituisce almeno un record
- **Esempio:** *selezionare fornitori con prodotti a prezzo < 20 senza includere il prezzo nel SELECT esterno*

ANY (SOME):

- True se uno qualsiasi dei valori della sottoquery soddisfa la condizione
- Esempio: prodotti con ordini > 90 in almeno un record

ALL:

- True solo se tutti i valori della sottoquery soddisfano la condizione
- **Esempio:** *prodotti per cui in order_details quantità = 10 per tutti i record collegati*

CASE e ORDER BY con CASE:

- **CASE:** valutazione sequenziale di condizioni; usa la prima vera
- Può restituire stringhe/valori calcolati
- Combinabile con ORDER BY per ordinamenti condizionali
- **Gestione NULL:** IFNULL/COALESCE per valori alternativi quando NULL

Schema relazionale e chiavi: **Chiave primaria e chiave esterna**

- **Primary Key:** identificatore univoco, indicizzato, nessun duplicato
- **Foreign Key:** riferimento a record di un'altra tabella
- Vincoli per prevenire operazioni "illegali" e riferimenti orfani

Relazioni molti-a-molti:

- **Esempio:** *recita (attore-film)*
- Tabella di giunzione con ripetizioni valide delle FK
- Evitare colonne multiple per riferimenti; usare tabelle ponte

DDL e gestione database: CREATE/DROP DATABASE

- Creazione ed eliminazione del database
- DROP DATABASE elimina tutte le tabelle contenute

CREATE TABLE e tipi di dato: *Definizione colonne e data type*

- Stringhe:

- **VARCHAR:** fino a ~65.000 caratteri
- **MEDIUMTEXT e LONGTEXT:** fino a ~16.000 e ~4 miliardi di caratteri
- Scelta tipo influisce su partizione/spazio su disco
- **ENUM:** insieme di valori predefiniti
- **Numerici: INT, FLOAT**
- **Date:** formato **YYYY-MM-DD**

DROP TABLE vs TRUNCATE TABLE:

- **DROP TABLE:** elimina tabella e contenuti
- **TRUNCATE TABLE:** svuota dati, mantiene schema

ALTER TABLE:

- **Modifica schema:** aggiunta/rimozione colonne, cambio tipo, ecc.

Vincoli (Constraints) e policy

Vincoli comuni:

- **NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT**
- **Definiti a livello di schema;** impediscono insert/update non validi

Scelte di implementazione controlli:

- Due approcci: lato DB (DBMS) e/o lato codice (API)
- In aziende grandi: *entrambi*
- In progetti rapidi/prototipi: scegliere uno, in base a esigenze/comfort

Permessi e ruoli nel DBMS:

- **Accessi differenziati per utente**
- **Guest:** spesso nessuna lettura
- **Lettura sola per utenti base su tabelle specifiche**
- **Operazioni critiche (CREATE/DROP DB) riservate agli admin**
- **Nota:** *problemi comuni di connessione (es. MySQL non connesso al DB)*

INSERT...SELECT e viste | INSERT INTO ... SELECT:

- *Copia dati da tabella2 a tabella1*
- *Unisce semantica di INSERT e SELECT*

VIEW:

- *Tabella virtuale basata sul risultato di una query*
- **Non memorizza dati; struttura persistente, contenuto dinamico**
- **Vantaggi:** *riuso, performance (meno parsing/ottimizzazione ripetuta), semplificazione accesso*
- **Operazioni: CREATE VIEW, CREATE OR REPLACE VIEW, DROP VIEW**
- **Esempio:** view "clienti_del_Brasile" con CustomerID e ContactName filtrati su Country = 'Brazil'

- Esercizio proposto (VIEW su città italiane) | Obiettivo:

- *Creare una view su "città" con tutte le città italiane con popolazione ≥ 100.000*

- Indicazioni:

- Usare CREATE VIEW ... AS SELECT ...
- SELECT con WHERE Country = 'Italy' AND Population ≥ 100000
- Verifica entro 10 minuti dall'assegnazione